

Laravel React Sample App - API client update for pagination



By Andrea Dao

DEVELOPER DOCUMENTATION SPECIALIST

Paginating data helps you manage user interactions or handle batch information when fetching data from the BigCommerce APIs. With this update, you can focus on building your React components while the API client handles the complexity of pagination and API interactions.

Our [Laravel React Sample app](#) now includes examples for two use cases of pagination:

- Managing UI interactions in a paginated inventory table with interactive controls
- Retrieving large datasets for inventory reports and CSV exports

The following article explores how the API client handles pagination. We have updated the client-side portion of the API client to either handle or pass pagination data through its service layers, depending on the use case.

The figure below outlines the overall structure of the sample app and highlights the client-side portion that we updated in the example implementation.

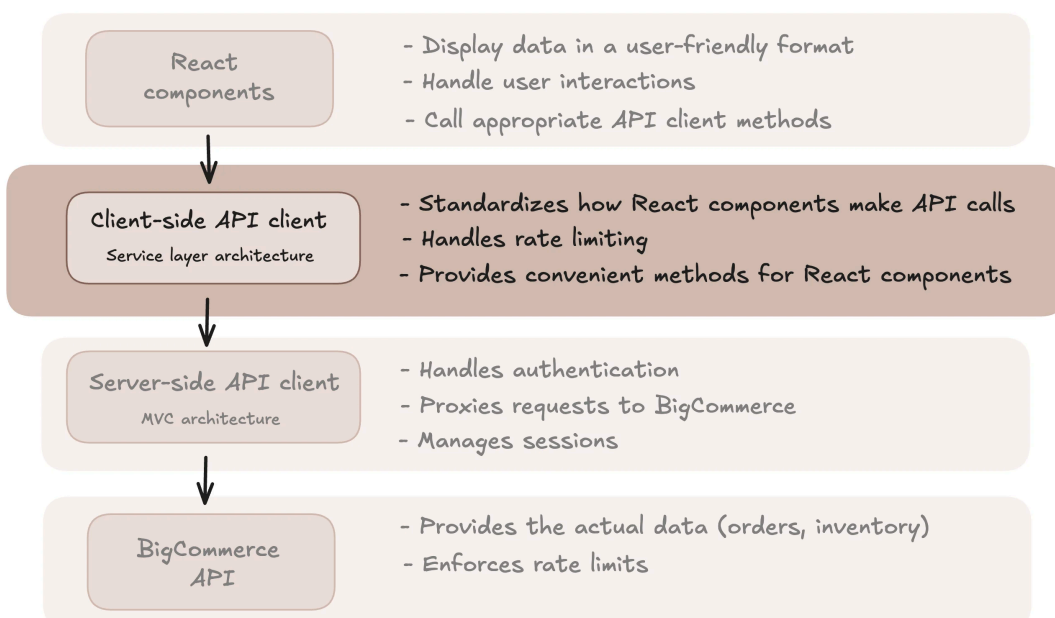


Figure 1: App structure. The API client that makes it easier to work with the BigCommerce API has a split architecture.

API client update - Javascript service layers

Our updates to the API client allow you to handle pagination in the React component for user interaction or in the service layer for bulk operations.

Component Layer Pagination

Pagination in the component layer refers to handling and displaying paginated data within the React component (i.e. UI-driven pagination).

Component layer pagination is useful for:

- User navigation
- Page controls
- Visual feedback
- UI state management

For component layer pagination, you can use [ApiService](#), which uses the following approach:

- Accepts page and limit parameters
- Returns pagination metadata to the React component with each response
- Lets the React component handle page navigation

Overall, ApiService fetches small subsets of data at a time, allowing users to navigate through the data set using pagination controls.

Service Layer Pagination

Service layer pagination refers to handling pagination entirely within the service layer, where the service iterates through all available pages of data automatically, making sequential API requests until all pages are retrieved.

Service layer pagination is useful for:

- Batch processing
- Report generation
- Data exports
- Large dataset operations

For scenarios requiring complete data retrieval, you can use [BatchApiService](#) which automates pagination with the following approach:

- Fetches multiple pages of data automatically, using pagination data to reduce server load
- Handles rate limits between requests
- Concatenates all results into a single response to return to the React Component

Overall, BatchApiService abstracts pagination complexity away from React components, rather than having them manage page numbers, limits, or multiple API calls themselves.

Both service layers include built-in protection for rate limits via throttling, which helps prevent API rate limit errors by introducing automatic delays.

The figure below summarizes the similarities and differences between the two service layers:

FEATURE	API SERVICE	BATCH API SERVICE
Similarities		
Client-side service layers in the API client	--	--
Create consistent interfaces for UI components to make API calls	--	--
Differences		
Use Case	Component layer pagination for user interactions, single-page fetching	Service layer pagination for batch operations
Collaboration with UI Component	Works interactively, with the UI requesting specific pages as needed	Works independently to fetch all pages at once
Data flow	UI request → ApiService → single API call → UI update → user interaction → repeat	Single request → multiple API calls → complete dataset → final UI update
Memory management	Only holds one page of data at a time	Accumulates all data before returning
User experience	Shows data immediately with interactive pagination	Shows loading state until all data is retrieved

Example Implementation

Component Layer Pagination

Inventory.jsx displays a paginated table of inventory items, allowing users to browse through the data with interactive page controls. The collaboration between Inventory.jsx and ApiService.js creates a memory-efficient interface for browsing large datasets.

1. Inventory.jsx uses ApiService.js to fetch a single page of 5 inventories at a time
2. Inventory.jsx uses the pagination data that ApiService.js returns for UI interactions.

0:00 / 0:03

Service Layer Pagination

Reports.jsx provides a user interface to export the complete inventory dataset as a CSV file. The collaboration between Reports.jsx and BatchApiService.js creates a seamless interface for bulk operations like CSV exports and report generation.

1. Reports.jsx uses BatchApiService.js to fetch all inventories in one operation
2. Reports.jsx uses the complete dataset that BatchApiService.js returns for generating CSV exports

0:00 / 0:04

Get started today!

Ready to dive into the Laravel React Sample app? Check out the [example implementation](#) from this blog and kickstart your project today! Reach out to devrel@bigcommerce.com if you have questions or feedback.

 Need help? [Join our BigCommerceDevs Slack](#)

 Find endpoints quickly in the [API reference](#)

 Stay up to date with [Release notes](#)

 Visit [BigCommerce home](#)



Everything you need for growth, all from one modern platform.



© Copyright 2003 - 2025 BigCommerce Pty. Ltd. — [Contact Us](#) [Careers](#) [Terms of Service](#) [Privacy Policy](#)